
Assignment Part 2: The .NET Built-in Delegates (Action, Func, Predicate)

Course Module: Advanced C# Concepts – Delegates & Functional Paradigms

Objective

Now that you understand how delegates work under the hood, it is time to write modern C#. In the real world, you rarely declare custom delegates. Instead, .NET provides three generic built-in delegates that can take up to 16 parameters.

Your task is to refactor our Aviation System to completely eliminate our custom delegates (`DiscountCalculator`, `NotificationSender`, etc.) and replace them with the built-in ones.

Task 1: Action<T> — The Void Operator

The `Action` delegate represents a method that takes parameters but returns `void`. It is perfect for logging, printing, or sending notifications.

- 1 Delete your custom `NotificationSender` delegate.
- 2 Create a method `BroadcastNotification(FlightTicket ticket, Action<FlightTicket> notificationAction)`.
- 3 In your `Main`, call this method and pass a lambda expression that prints: `[System Log] Processing ticket for {PassengerName}`.

Task 2: Predicate<T> — The Filter

The `Predicate` delegate represents a method that takes ONE parameter and strictly returns a `boolean`. It is the standard way to write conditions and filters.

- 1 Create a method `FilterTickets(List<FlightTicket> tickets, Predicate<FlightTicket> condition)`.
- 2 Inside the method, iterate over the tickets and return a new `List<FlightTicket>` containing only the tickets that pass the condition.
- 3 In your `Main`, use this method with a lambda expression to retrieve and print only the delayed flights.

Task 3: Func<T, TResult> — The Transformer

The `Func` delegate represents a method that takes parameters and returns a value. The LAST generic type always represents the return type.

- 1 Delete your custom `DiscountCalculator` delegate.

-
- 2 Modify your `ProcessTickets` method to accept a `Func<FlightTicket, decimal>` instead of the custom delegate.
 - 3 Create a new method `GetTicketSummaries(List<FlightTicket> tickets, Func<FlightTicket, string> formatter)`.
 - 4 In your `Main`, use `GetTicketSummaries` to transform your list of tickets into a list of strings formatted as: `"{TicketId} - {PassengerName}"`, and print them. (This is exactly how LINQ's `.Select()` works!)

End of Assignment — Please submit your completed solution as a single .cs file or a zipped Visual Studio project.